Cognivia GmbH

Entwicklungsumgebung

Study-Planner-Anwendung

Stand:	02.04.2026
Auftraggeber:	InnoWeb Solutions GmbH Friedrich-List-Platz 1 01069 Dresden kontakt@innowebsolutions.de
Auftragnehmer:	Cognivia GmbH Moritzstraße 15 01097 Dresden
Ansprechperson:	Projektteam „Study-Planner“ kontakt@cognivia.de www.cognivia.de
Autoren:	Yara Dalloul Yasamin Habibollahi Dana Lenzig Luise Schäffel Ina Savina

Inhaltsverzeichnis

1 Einleitung....1
2 Bewertungsmethode und Vergleichskriterien....1
3 Toolvergleich nach Kategorien....3
    3.1 Versionierung....3
    3.2 UML-Tool mit Quellcodegenerierung....4
    3.3 Build-Tool....5
    3.4 Prototyping-Tool der Benutzerschnittstelle....6
    3.5 IDE / Editor....7
    3.6 Test-Automatisierung....8
    3.7 KI-Tools....9
    3.8 Dokumentationstool....10
    3.9 Obfuscator....11
    3.10 Codeconventions....12
    3.11 Kollaborationstool....13
4 Zusammenfassung der gewählten Entwicklungsumgebung....14
5 Quellen....15

1 Einleitung

Dieses Dokument legt die Entwicklungsumgebung der Cognivia GmbH für die Umsetzung der Study-Planner-Anwendung fest. Das Projekt wird im Auftrag der InnoWeb Solutions GmbH durchgeführt, die eine Desktop-Anwendung zur Planung und Verwaltung des Studienalltags entwickeln lassen möchte.

Ziel ist es, geeignete Werkzeuge für Entwicklung, Modellierung, Build, Test, Dokumentation und Zusammenarbeit auszuwählen und einheitlich im Projekt einzusetzen. Für jede Tool-Kategorie werden mehrere mögliche Werkzeuge betrachtet und anhand festgelegter Kriterien miteinander verglichen. Auf dieser Basis wird eine Auswahl getroffen und begründet.

Zusätzlich werden verbindliche Codekonventionen definiert, um eine konsistente Codebasis und eine effiziente Zusammenarbeit im Team sicherzustellen.

2 Bewertungsmethode und Vergleichskriterien

Die Bewertung erfolgt mithilfe eines Punktesystems von 1 bis 5 Punkten:

- 5 Punkte - sehr gut geeignet
- 4 Punkte - gut geeignet
- 3 Punkte - mittel geeignet / neutral
- 2 Punkte - eher ungeeignet
- 1 Punkt - ungeeignet

Die Gesamtbewertung eines Tools ergibt sich aus der Summe der vergebenen Punkte. In der Regel wird das Tool mit der höchsten Punktzahl ausgewählt. Falls mehrere Werkzeuge ähnlich bewertet werden oder praktische Gründe vorliegen, kann auch bewusst eine andere Entscheidung getroffen oder eine Kombination mehrerer Tools verwendet werden.

Für den Vergleich werden verschiedene Kriterien verwendet. Die nachfolgend aufgeführten Kriterien stellen übergeordnete Bewertungsaspekte dar, die in mehreren Tabellen wiederkehren oder thematisch zusammengefasst werden können. Je nach Tool-Kategorie werden zusätzlich spezifische Kriterien verwendet, die auf die jeweiligen Anforderungen zugeschnitten sind. Diese werden, falls nötig, unter den entsprechenden Tabellen erklärt.

Für den Vergleich werden folgende Kriterien verwendet:

Verbreitung / Standard

Bewertet wird, wie etabliert das Tool im jeweiligen Einsatzbereich ist und ob es häufig in Projekten oder im industriellen Umfeld verwendet wird.

Eignung für das Projekt

Bewertet wird, ob das Tool die Anforderungen der geplanten Anwendung sowie das gewählte Vorgehensmodell sinnvoll unterstützt.

Funktionsumfang

Bewertet wird, ob alle benötigten Funktionen vorhanden sind, die für die jeweilige Tool-Kategorie erforderlich sind.

Bedienbarkeit

Bewertet wird, wie übersichtlich und intuitiv das Tool nutzbar ist und ob typische Aufgaben schnell umgesetzt werden können.

Lernaufwand
Bewertet wird, wie viel Einarbeitungszeit erforderlich ist, um das Tool effektiv im Projekt einsetzen zu können.

Integration / Kompatibilität
Bewertet wird, wie gut sich das Tool mit anderen eingesetzten Werkzeugen kombinieren lässt und ob ein reibungsloser Workflow möglich ist.

Kollaboration / Teamfähigkeit
Bewertet wird, ob das Tool gemeinsames Arbeiten unterstützt, beispielsweise durch gemeinsame Bearbeitung, zentrale Verwaltung oder Abstimmungsfunktionen.

Kosten / Lizenz
Bewertet wird, ob das Tool wirtschaftlich sinnvoll einsetzbar ist und ob Lizenzkosten im Verhältnis zum Nutzen stehen. Kostenlose oder frei verfügbare Werkzeuge werden dabei entsprechend positiver bewertet.

Stabilität / Zuverlässigkeit
Bewertet wird, ob das Tool im praktischen Einsatz zuverlässig arbeitet und regelmäßig gepflegt wird.

Erweiterbarkeit / Community
Bewertet wird, ob zusätzliche Erweiterungen verfügbar sind und ob eine aktive Community Unterstützung und Dokumentation bietet.

3 Toolvergleich nach Kategorien

3.1 Versionierung

Zur Verwaltung des Quellcodes und der Projektdokumente wird ein Versionsverwaltungssystem eingesetzt. Es ermöglicht nachvollziehbare Änderungen, paralleles Arbeiten im Team (Branches) und die Wiederherstellung früherer Stände.

Kriterium	Git	Subversion (SVN)	Mercurial (hg)
Verbreitung / Standard	**5** Industriestandard	**3** Ältere Projekte	**2** Kaum verbreitet
Arbeitsmodell *	**5** Verteilt, lokale Branches, offline möglich	**2** Zentral, Branching und Merge komplizierter	**4** Verteilt, weniger Tools
Team-Workflow	**5** Pull- und Merge-Requests, Branching	**3** Branching umständlich	**3** Branch, aber weniger Collaboration-Tools
Plattform / Integration	**5** GitLab, GitHub, IDE-Integration	**4** Weniger moderne Features	**3** Deutlich weniger Plattform-Support
Lernaufwand	**3** Komplexer	**4** Einfacher, weniger Konzepte	**3** Komplexer
Kosten	**5** Kostenlos	**5** Kostenlos	**5** Kostenlos
Gesamt	**28**	**21**	**20**

Ausgewähltes Tool: **Git** (GitLab)

Git wird ausgewählt, da es die höchste Gesamtbewertung erreicht und als Industriestandard weit verbreitet ist. Das verteilte Arbeitsmodell unterstützt paralleles Arbeiten im Team durch Branching und Merging. In Kombination mit GitLab bietet das Tool zudem eine sehr gute Integration von Repository, Issue-Tracking und Merge Requests, wodurch die Zusammenarbeit im Projekt effizient organisiert werden kann.

* Arbeitsmodell: Bewertet wird, wie Änderungen organisiert sind und ob Entwickler unabhängig lokal arbeiten können oder auf eine zentrale Verwaltung angewiesen sind.

3.2 UML-Tool mit Quellcodegenerierung

Für Modellierung und Dokumentation werden UML-Diagramme erstellt, z. B. Use Cases, Klassendiagramme, Sequenzen. Ein geeignetes UML-Tool soll die Erstellung, Pflege und Exportierbarkeit der Modelle unterstützen. Optional ist eine (teilweise) Quellcodegenerierung hilfreich, ist aber nicht zwingend für alle Diagrammtypen erforderlich.

Kriterium	Papyrus (Eclipse)	StarUML	Visual Paradigm
UML-Unterstützung	**5** UML2 vollständig, professionell	**4** Weniger vollständig	**5** Sehr umfangreich
Quellcode-generierung *	**4** Möglich über Eclipse Plugins, weniger komfortabel	**3** Nur teilweise / Extensions	**5** Integriert
Bedienbarkeit	**3** Komplex	**4** Intuitive UI	**4** Gut, aber viele Funktionen
Export (PDF/PNG)	**4** Möglich, teilweise umständlich	**4** Solide	**5** Sehr viele Exportoptionen
Kosten / Lizenz	**5** Kostenlos	**2** Kostenpflichtig	**2** Kostenpflichtig
Integration ins Projekt	**5** Direkt in Eclipse	**4** Dateibasiert	**4** Dateibasiert
Gesamt	**26**	**21**	**25**

Ausgewähltes Tool: **Papyrus**

Papyrus erhält die höchste Gesamtbewertung und wird daher für das Projekt ausgewählt. Das Tool bietet eine sehr gute UML-Unterstützung, ist vollständig kostenlos nutzbar und lässt sich direkt in Eclipse integrieren. Dadurch können Modelle strukturiert verwaltet und problemlos in den Entwicklungsprozess eingebunden werden. Zudem ist eine optionale Quellcodegenerierung über das Eclipse-Ökosystem möglich, wodurch Papyrus eine geeignete Lösung für die Modellierung im Projekt darstellt.

* Quellcodegenerierung: Beschreibt, ob aus UML-Modellen automatisch Programmcode erzeugt werden kann.

3.3 Build-Tool

Ein Build-Tool dient dazu, das Projekt reproduzierbar zu bauen (Kompilieren, Abhängigkeiten verwalten, Tests ausführen und Artefakte erzeugen). Damit kann das Team die Anwendung unabhängig von der lokalen IDE konsistent erstellen und testen.

Kriterium	Maven	Gradle	Ant
Standard für Java	**5** Langjähriger Standard, weit verbreitet	**4** Modern, aber nicht überall Standard	**2** Veraltet
Dependency-Management *	**5** Zentrale Features	**5** Sehr gut	**1** Nur manuell
Konfiguration	**4** XML strukturiert, gut lesbar	**4** Flexibel (Groovy/ Kotlin)	**2** Wenig Struktur, viel manuell
Lernaufwand	**4** Einfache Standardstruktur	**3** DSL muss gelernt werden	**3** Simpel, aber manuell
Plugins	**5** Sehr groß	**5** Sehr groß	**2** Begrenzt
CI-Integration * (GitLab)	**5** GitLab CI Standard	**5** Sehr gut	**3** Möglich, aber mehr Aufwand
Gesamt	**28**	**26**	**13**

Ausgewähltes Tool: **Maven**

Für das Projekt wird Maven eingesetzt, da es im Vergleich die höchste Gesamtpunktzahl erzielt. Maven bietet eine strukturierte Projektorganisation und ein zuverlässiges Dependency-Management, wodurch Abhängigkeiten zentral verwaltet werden können. Außerdem unterstützt das Tool eine einfache Integration in Continuous-Integration-Prozesse, sodass Builds und Tests automatisiert und reproduzierbar durchgeführt werden können.

* Dependency Management: Verwaltung externer Bibliotheken und deren Versionen, die im Projekt benötigt werden.

* CI-Integration: Einbindung des Build-Tools in automatisierte Build- und Testprozesse, z. B. über GitLab CI.

3.4 Prototyping-Tool für die Benutzerschnittstelle

Für die frühe Abstimmung von Layout und Bedienabläufen werden UI-Prototypen erstellt. Ein Prototyping-Tool soll schnelle Mockups ermöglichen, damit Feedback früh eingeholt und Änderungen einfach umgesetzt werden können.

Kriterium	Figma	Pencil Project	draw.io
Geschwindigkeit für Mockups *	**5** Speziell für UI-Prototyping	**4** Gut für einfache Mockups	**3** Eher Diagrammtool
Kollaboration	**5** Echtzeit-Zusammenarbeit, gemeinsam	**2** Keine echte Kollaboration	**4** Teilen möglich, aber kein UX-Fokus
UX/UI-Fokus *	**5** Speziell für UI/UX entwickelt	**4** UI-Bibliotheken vorhanden	**3** Eher für Diagramme
Export (PNG/PDF)	**5** PNG, PDF, SVG	**4** Standard Export	**5** Sehr viele Exportoptionen
Lernaufwand	**4** Etwas Einarbeitung	**5** Sehr einfach	**5** Sehr intuitiv
Kosten / Lizenz	**5** Kostenlos	**5** Kostenlos	**5** Kostenlos
Gesamt	**29**	**24**	**25**

Ausgewähltes Tool: **Figma**

Für das Prototyping der Benutzeroberfläche wird Figma eingesetzt, da es im Vergleich die meisten Punkte erzielt. Mit Figma lassen sich Benutzeroberflächen schnell visualisieren und Änderungen flexibel anpassen. Zudem unterstützt das Tool die kollaborative Arbeit, sodass mehrere Teammitglieder gleichzeitig an Entwürfen arbeiten und Feedback direkt einfließen lassen können.

* Geschwindigkeit für Mockups: Bewertet wird, wie schnell einfache Oberflächenentwürfe erstellt und angepasst werden können.

* UX/UI-Fokus: Bewertet wird, wie gut das Tool speziell auf die Gestaltung von Benutzeroberflächen und Benutzerführung ausgelegt ist.

3.5 IDE / Editor

Für die Implementierung der Java-Anwendung wird eine Entwicklungsumgebung (IDE) benötigt. Sie soll das Schreiben, Strukturieren und Debuggen des Codes unterstützen sowie die Integration von Build- und Test-Tools erleichtern.

Kriterium	IntelliJ IDEA	Eclipse	VS Code
Java-Unterstützung	**5** Sehr starke Java-Analyse	**5** Etablierte Java-IDE	**4** Über Extensions
Bedienbarkeit	**5** Moderne UI	**3** Eher komplex	**5** Sehr intuitiv
Refactoring / Code-Analyse *	**5** Umfangreich	**4** Gute Standard-Refactorings	**3** Begrenzt
Maven-Integration	**5** Vollständig integriert	**4** Integration über m2e Plugin	**4** Extensions nötig
Git-Integration	**5** Umfassende Git-Funktionen	**4** Git über EGit verfügbar	**4** Integrierte Git-Grundfunktion
Kosten / Lizenz	**4** Community-Version kostenlos, Ultimate kostenpflichtig	**5** Kostenlos	**5** Kostenlos
Lernaufwand	**4** Etwas Einarbeitung	**3** Komplexe Oberfläche	**5** Sehr leicht
Gesamt	**33**	**28**	**30**

Ausgewähltes Tool: **Eclipse** und **VS Code**

Für das Projekt werden Eclipse und Visual Studio Code kombiniert eingesetzt. Eclipse wird als Haupt-IDE für die Java-Entwicklung verwendet, insbesondere aufgrund der Integration mit Papyrus und Maven. Visual Studio Code wird ergänzend für leichte Bearbeitungen, Markdown-Dokumentation und schnelle Änderungen genutzt. Diese Kombination verbindet eine leistungsfähige Java-IDE mit einer flexiblen, leichtgewichtigen Entwicklungsumgebung.

* Refactoring / Code-Analyse: Unterstützt das strukturierte Umgestalten des Codes sowie automatische Prüfungen zur Verbesserung von Qualität und Lesbarkeit.

3.6 Test-Automatisierung

Für die Qualitätssicherung werden automatisierte Tests eingesetzt. Sie dienen dazu, Kernfunktionen reproduzierbar zu prüfen und Regressionen bei Änderungen früh zu erkennen. Insbesondere sollen zentrale Teile der Fachlogik, z. B. Verwaltung von Modulen, Terminen, Lernblöcken und Filterlogik, getestet werden.

Kriterium	JUnit 5	TestNG	Spock
Standard im Java-Umfeld	**5** Standard für Java-Unit-Tests	**3** Verbreitet, aber weniger genutzt	**3** Eher spezialisiert (Groovy)
Integration in Maven	**5** Standardmäßig in Maven integriert	**5** Sehr gut integrierbar	**4** Möglich, aber zusätzliche Konfigurationen
Integration in Eclipse	**5** Direkt integriert, sofort nutzbar	**4** Plugin nötig	**3** Zusätzliche Plugins erforderlich
Lernaufwand	**4** Einfache Struktur	**3** Komplexer	**2** Groovy-Syntax muss gelernt werden
Funktionsumfang	**4** Ausreichend	**5** Erweiterte Features	**5** Sehr expressive Tests möglich
Community / Dokumentation	**5** Sehr große Community, viele Beispiele	**4** Gute Dokumentation vorhanden	**3** Kleinere Community, spezieller Einsatz
Gesamt	**28**	**24**	**20**

Ausgewähltes Tool: **JUnit 5**

JUnit 5 wird ausgewählt, da es der De-facto-Standard für Unit-Tests im Java-Umfeld ist und die höchste Gesamtbewertung erreicht. Das Tool lässt sich sehr gut in Maven und Eclipse integrieren und ermöglicht eine einfache Erstellung automatisierter Tests. Durch den geringen Lernaufwand und die große Community eignet sich JUnit 5 besonders für eine stabile und wartbare Testautomatisierung im Projekt.

3.7 KI-Tools

KI-Tools können das Team bei der Entwicklung unterstützen, z. B. beim Formulieren von Texten, beim Erstellen kleiner Code-Snippets, beim Debugging sowie bei der Ideenfindung für Tests und Dokumentation. Die Verantwortung für fachliche Richtigkeit, Datenschutz und die finale Umsetzung bleibt jedoch beim Projektteam.

Kriterium	ChatGPT	Claude	GitHub Copilot
Unterstützung für Code	**5** Dialogbasiert	**5** Dialogbasiert	**5** Inline-Codevorschläge
Unterstützung für Text/Dokumentation	**5** Sehr gut für Dokumentation und Strukturierung	**5** Sehr gut für Dokumentation und Strukturierung	**3** Fokus auf Code, weniger auf Dokumentation
Lernaufwand	**5** Sofort nutzbar	**5** Sofort nutzbar	**4** Einrichtung und Nutzung im Editor erforderlich
Integration in IDE	**3** Nutzung über Browser oder App	**3** Nutzung über Browser oder App	**5** Direkte Integration möglich
Kosten / Lizenz	**4** Abhängig vom gewählten Plan	**4** Abhängig vom gewählten Plan	**3** Kostenpflichtig
Datenschutz / Kontrolle *	**3** Abhängig von Nutzung und Eingabe	**3** Abhängig von Nutzung und Eingabe	**3** Abhängig von GitHub-Einstellungen
Gesamt	**25**	**25**	**23**

Ausgewählte Tools: **ChatGPT** und **Claude**

ChatGPT und Claude werden im Projekt ergänzend eingesetzt, da beide Tools eine gute Unterstützung bei der Erstellung von Dokumentation, bei technischen Fragestellungen und beim Debugging bieten. Sie können außerdem bei der Formulierung von Aufgabenbeschreibungen, Testfällen sowie bei der Strukturierung von Projektinhalten helfen. Die Verantwortung für fachliche Richtigkeit und die finale Umsetzung verbleibt beim Projektteam. Sensible Projektdaten werden nur abstrahiert oder in gekürzter Form in die KI-Tools eingegeben.

*Datenschutz / Kontrolle: Bewertet wird, inwieweit eingegebene Daten geschützt sind und wie stark die Nutzung der KI kontrolliert werden kann.

3.8 Dokumentationstool

Zur technischen Dokumentation des Projekts werden Werkzeuge eingesetzt, um Quellcode und Projektwissen nachvollziehbar zu beschreiben. Ziel ist eine leicht zugängliche Entwicklerdokumentation sowie eine kurze Anwenderdokumentation für den Einsatz der Anwendung.

Kriterium	Javadoc	Doxygen	Markdown
Eignung für Java-Code	**5** Standard für Java-API-Dokumentation	**4** Unterstützt Java, aber nicht spezialisiert	**3** Nur indirekt für Code-Doku geeignet
Automatische Generierung	**5** Automatisch	**5** Automatisch	**1** Manuell
Lernaufwand	**4** Gering	**3** Mehr Konfiguration erforderlich	**5** Gering
Ausgabeformat	**4** HTML-Ausgabe standardgemäß	**5** Mehrere Formate (HTML, PDF)	**4** Flexibel je nach Tool
Integration ins Projekt	**5** Gut in Maven und IDE integrierbar	**4** Integration möglich, aber zusätzlicher Aufwand	**5** Direkt im Repository nutzbar
Nutzung im Team	**5** Sehr gut für Entwickler-dokumentation	**4** Breiter Einsatz möglich	**5** Ideal für Projektübersicht
Gesamt	**28**	**25**	**23**

Ausgewählte Tools: **Javadoc** und **Markdown**

Javadoc wird für die automatische Entwicklerdokumentation direkt aus dem Java-Quellcode genutzt und ist im Java-Umfeld der Standard. Zusätzlich wird eine kurze Projektdokumentation in Markdown (README) gepflegt, um Setup, Build-Schritte und Projektstruktur für das Team schnell auffindbar zu dokumentieren. Diese Kombination ermöglicht sowohl eine API-nahe Dokumentation als auch eine leicht lesbare Projektübersicht im Repository.

3.9 Obfuscator

Ein Obfuscator dient dazu, den kompilierten Code (z. B. Java-Bytecode) zu verschleiern, um Reverse Engineering zu erschweren und interne Implementierungsdetails zu schützen.

Kriterium	ProGuard	yGuard	DashO
Verbreitung im Java-Umfeld	**5** Standard-Tool	**3** Weniger verbreitet	**3** Eher kommerzieller Einsatz
Kosten / Lizenz	**5** Kostenlos und Open Source	**5** Kostenlos	**2** Kostenpflichtig
Integration in Build (Maven/Gradle)	**5** Sehr gute Maven-Integration	**4** Integration möglich, etwas mehr Aufwand	**5** Gute Integration vorhanden
Funktionsumfang	**5** Obfuscation und Shrinking integriert	**4** Grundlegende Funktionen	**5** Umfangreiche Features
Lernaufwand	**3** Moderate Konfiguration notwendig	**3** Moderate Konfiguration notwendig	**3** Moderate Konfiguration notwendig
Eignung für das Projekt	**5** Ausreichend für Projekt	**4** Geeignet, aber weniger verbreitet	**3** Überdimensional für Projekt
Gesamt	**28**	**23**	**21**

Ausgewähltes Tool: **ProGuard**

ProGuard wird verwendet, da es im Vergleich die höchste Gesamtbewertung erreicht und im Java-Umfeld weit verbreitet ist. Das Tool bietet Funktionen zur Verschleierung und Optimierung des kompilierten Codes und lässt sich gut in den Build-Prozess integrieren. Durch den ausreichenden Funktionsumfang und die kostenfreie Nutzung eignet sich ProGuard für die Anforderungen des Projekts.

3.10 Codeconventions

Einheitliche Codekonventionen stellen sicher, dass der Quellcode im Team konsistent, lesbar und wartbar bleibt. Sie betreffen u. a. Formatierung, Namensgebung, Strukturierung von Klassen sowie Dokumentationsregeln.

Kriterium	Google Java Style Guide	Oracle Code Conventions	Team-eigene Konvention
Verbreitung / Standard	**5** Industriestandard	**4** Bekannt, aber weniger genutzt	**2** Nur projektintern gültig
Detailgrad *	**5** Sehr detaillierte Regeln	**3** Weniger umfangreich	**3** Abhängig vom Team
Tool-Unterstützung	**5** Formatter und Checks verfügbar	**4** Teilweise unterstützt	**2** Keine automatische Unterstützung
Konsistenz im Team	**5** Klare, einheitliche Regeln	**4** Konsistent nutzbar	**3** Risiko unterschiedlicher Nutzung
Lernaufwand	**3** Viele Regeln	**4** Weniger komplex	**3** Abhängig von Definition
Gesamt	**23**	**19**	**13**

Ausgewählte Konvention: **Google Java Style Guide**

Der Google Java Style Guide wird verwendet, da er weit verbreitet ist und eine klare, einheitliche Struktur vorgibt. Zudem wird er von vielen Tools unterstützt, wodurch die Konsistenz im Team verbessert wird.

* Detailgrad: Bewertet wird, wie umfangreich und genau die definierten Regeln für Formatierung und Struktur des Quellcodes sind.

3.11 Kollaborationstool

Für die Teamarbeit werden Werkzeuge benötigt, um Aufgaben, Bugs und Fortschritt transparent zu dokumentieren und die Abstimmung im Projekt effizient zu organisieren. Ziel ist eine klare Nachvollziehbarkeit von Zuständigkeiten, Status und Änderungen.

Kriterium	GitLab Issues	WhatsApp / E-Mail	Präsenzabstimmung
Aufgabenverwaltung	**5** Zuständigkeiten und Status möglich	**2** Nicht gut strukturiert	**3** Nur mit Protokoll nutzbar
Nachvollziehbarkeit / Historie *	**5** Komplette Änderungshistorie	**2** Unübersichtlich	**2** Dokumentation erforderlich
Integration mit Code	**5** Direkte Verknüpfung mit Commits	**1** Keine Integration	**1** Keine Integration
Schnelle Kommunikation	**3** Eher asynchron	**5** Sehr schnell	**4** Direkte Kommunikation möglich
Verbindliche Entscheidungen *	**5** Dokumentiert und nachvollziehbar	**3** Gefahr von Informationsverlust	**4** Verbindlich bei Protokoll
Eignung für das Projekt	**5** Zentrales Organisationstool	**3** Ergänzend	**3** Sinnvoll für Abstimmung
Gesamt	**28**	**16**	**17**

Ausgewählte Tools: **GitLab Issues, WhatsApp** und **Präsenzabstimmung**

GitLab Issues werden als zentrales Werkzeug für Aufgaben, Bugs und Fortschritt genutzt, da sie direkt mit Repository, Commits und Merge Requests verknüpft werden können. Dadurch bleibt die Projektarbeit transparent und nachvollziehbar. Für die Abstimmung im Team werden regelmäßige Präsenztreffen genutzt, um offene Punkte, Prioritäten und nächste Schritte effizient zu klären. Kurzfristige Kommunikation erfolgt ergänzend über WhatsApp. Verbindliche Entscheidungen und Aufgaben werden anschließend in GitLab Issues dokumentiert.

* Nachvollziehbarkeit / Historie: Bewertet wird, ob Änderungen und Entscheidungen dokumentiert und später nachvollzogen werden können.

* Verbindliche Entscheidungen: Bewertet wird, ob Absprachen festgehalten und für alle Teammitglieder verbindlich sind.

4 Zusammenfassung der gewählten Entwicklungsumgebung

Die nachfolgende Übersicht fasst die für das Projekt ausgewählte Toolkette der Cognivia GmbH zusammen. Sie bildet die verbindliche Grundlage für die Umsetzung der Study-Planner-Anwendung.

Versionierung:	**Git** (GitLab)
UML-Tool:	**Papyrus**
Build-Tool:	**Maven**
Prototyping:	**Figma**
IDE / Editor:	**Eclipse** und **VS Code**
Test-Automatisierung:	**JUnit 5**
KI-Tools:	**ChatGPT** und **Claude**
Dokumentation:	**Javadoc** und **Markdown**
Obfuscator:	**ProGuard**
Codeconventions:	**Google Java Style Guide**
Kollaborationstool:	**GitLab Issues, WhatsApp** und **Präsenzabstimmung**

5 Quellen

Die Bewertung der Werkzeuge basiert auf den offiziellen Dokumentationen und Webseiten der jeweiligen Tools sowie auf der Projekterfahrung des Teams. Die finale Auswahl und Bewertung erfolgte durch das Projektteam.

Git: https://git-scm.com
GitLab: https://about.gitlab.com

Papyrus: https://www.eclipse.org/papyrus/
StarUML: https://staruml.io
Visual Paradigm: https://www.visual-paradigm.com

Maven: https://maven.apache.org
Gradle: https://gradle.org
Apache Ant: https://ant.apache.org

Figma: https://www.figma.com
Pencil Project: https://pencil.evolus.vn
draw.io (diagrams.net): https://www.diagrams.net

Eclipse: https://www.eclipse.org
Visual Studio Code: https://code.visualstudio.com
IntelliJ IDEA: https://www.jetbrains.com/idea/

JUnit 5: https://junit.org/junit5/
TestNG: https://testng.org
Spock: https://spockframework.org

ChatGPT: https://chat.openai.com
Claude: https://claude.ai
GitHub Copilot: https://github.com/features/copilot

Javadoc: https://docs.oracle.com/javase/8/docs/technotes/tools/windows/javadoc.html
Doxygen: https://www.doxygen.nl
Markdown (GitHub Guide): https://guides.github.com/features/mastering-markdown/

ProGuard: https://www.guardsquare.com/proguard
yGuard: https://yworks.github.io/yGuard/
DashO: https://www.preemptive.com/products/dasho

Google Java Style Guide: https://google.github.io/styleguide/javaguide.html
Oracle Code Conventions: https://www.oracle.com/java/technologies/javase/codeconventions-contents.html